

Theoretical Guide

py.Sandu

Élvis Miranda, Leonardo Krauss & Rafael Alencar

1 DP

1.1 DP Patterns

DP applies when a problem has **overlapping subproblems** and **optimal substructure** (optimal solution uses optimal solutions to subproblems).

- **0/1 Knapsack** — $O(nW)$: each item used at most once. $dp[w] = \max(dp[w], dp[w - w_i] + v_i)$. Iterate w **downward** ($W \rightarrow w_i$).
- **Unbounded Knapsack** — $O(nW)$: each item reusable. Same recurrence but iterate w **upward** ($w_i \rightarrow W$).
- **Longest Common Subsequence (LCS)** — $O(nm)$:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{if } s[i] = t[j] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{otherwise} \end{cases}$$

- **Edit Distance** — $O(nm)$: min insertions/deletions/substitutions to turn s into t .

$$dp[i][j] = \min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + [s[i] \neq t[j]])$$

- **LIS** — $O(n \log n)$: maintain array **tails** where **tails[k]** = smallest tail of IS of length $k + 1$. For each element, binary search (**lower_bound**) for its position.
- **Interval DP** — $O(n^3)$: solve on subintervals $[l, r]$ using split point k .

$$dp[l][r] = \min_{k=l}^{r-1} (dp[l][k] + dp[k+1][r] + \text{cost}(l, r))$$

Classic: Matrix Chain Multiplication, Optimal BST, Burst Balloons.

- **Tree DP**: Root the tree; $dp[v]$ depends only on $dp[\text{children of } v]$. Process in post-order (leaves first). Classic: max independent set on tree, tree diameter, tree centroid.

- **Bitmask DP** — $O(2^n \cdot n)$: state is a subset of elements. $dp[\text{mask}]$ = optimal for the subset encoded by **mask**. Classic: TSP in $O(2^n \cdot n^2)$: $dp[\text{mask}][v] = \min_u (dp[\text{mask} \setminus \{v\}][u] + \text{dist}[u][v])$.
- **Digit DP**: Count integers in $[0, N]$ with a property. State: $(\text{position}, \text{tight}, \dots)$ where **tight** = whether current prefix equals the prefix of N .
- **DP on DAG**: Topologically sort the DAG; relax edges in order. Longest/shortest path in DAG is $O(V + E)$.
- **Prefix Sum DP**: $dp[i][j] = \sum_k dp[i-1][k]$ becomes $O(n^2)$ with prefix sums instead of $O(n^3)$. Generalize with 2D prefix sums for 2D states.

1.2 DP Optimizations

- **Space Optimization (Rolling Array)**: When $dp[i]$ depends only on $dp[i-1]$, keep only two rows. For knapsack: one 1D array suffices (iterate in the right direction). Reduces $O(nW)$ space to $O(W)$.
- **Convex Hull Trick (CHT)**: Optimizes recurrences of the form

$$dp[i] = \min_{j < i} (dp[j] + b[j] \cdot x[i])$$

where $b[j]$ is a slope and $x[i]$ is a query point. Maintain a lower convex hull of lines $y = b[j] \cdot x + dp[j]$; query in $O(1)$ (amortized) if $x[i]$ is monotone, or $O(\log n)$ with binary search. Reduces $O(n^2)$ to $O(n \log n)$ or $O(n)$.

- **Divide & Conquer Optimization**: Optimizes recurrences of the form

$$dp[i][j] = \min_{k \leq j} (dp[i-1][k] + C(k, j))$$

when the optimal split $\text{opt}(i, j)$ satisfies $\text{opt}(i, j) \leq \text{opt}(i, j+1)$ (*monotone opt*). Solve each layer using D&C on j : $T(n) = O(n \log n)$ per layer. Requires C to satisfy the **quadrangle inequality**: $C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$ for $a \leq b \leq c \leq d$.

- **Knuth–Yao Optimization:** Specialization for interval DP:

$$dp[l][r] = \min_{l \leq k < r} (dp[l][k] + dp[k+1][r] + C(l, r))$$

If C satisfies the quadrangle inequality, the optimal k is monotone: $\text{opt}[l][r-1] \leq \text{opt}[l][r] \leq \text{opt}[l+1][r]$. This reduces $O(n^3)$ to $O(n^2)$.

- **SMAWK / Divide & Conquer on Monge Matrices:** When the DP cost matrix is totally monotone (each row's minimum shifts right across rows), the row minima can be found in $O(n)$ with SMAWK or $O(n \log n)$ with D&C.
- **Memoization vs Tabulation:**
 - *Memoization* (top-down): recurse + cache. Simpler for sparse state spaces or when not all states are needed.
 - *Tabulation* (bottom-up): fill table iteratively. Usually faster in practice (no recursion overhead) and easier to optimize space.

2 Progressions

A progression is a sequence of numbers ordered according to a specific, logical rule or formula, where each term is derived from the previous one.

2.1 Special Sums

- Sum of first n natural numbers: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$
- Sum of first n odd numbers: $\sum_{i=1}^n (2i-1) = n^2$
- Sum of first n even numbers: $\sum_{i=1}^n 2i = n(n+1)$
- Sum of first n squares: $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$
- Sum of first n cubes: $\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2}\right)^2$
- Sum of first n squares of odd numbers: $\sum_{i=1}^n (2i-1)^2 = \frac{n(4n^2-1)}{3}$
- Sum of first n squares of even numbers: $\sum_{i=1}^n (2i)^2 = \frac{4n(n+1)(2n+1)}{6}$

2.2 Geometric Progression

General Term: $a_1 \times q^{n-1}$

Sum:

$$\sum_{i=1}^n a_1 q^{i-1} = a_1 \frac{1-q^n}{1-q}$$

Infinite Sum:

$$-1 < q < 1$$

$$\frac{a_1}{1-q}$$

2.3 Arithmetic Progression

A sequence where each term differs from the previous by a constant **common difference** r : $a_1, a_1 + r, a_1 + 2r, \dots$

General Term: $a_n = a_1 + (n-1)r$

Sum of first n terms:

$$S_n = \frac{n}{2}(a_1 + a_n) = n a_1 + \frac{n(n-1)}{2} r$$

Special case ($a_1 = 1, r = 1$):

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

3 Geometry

3.1 Lines & Polygons

- **Slope-Intercept:** $y = mx + c$
- **General Line Equation:** $Ax + By + C = 0$
- **Distance from point (x_0, y_0) to line $Ax + By + C = 0$:** $d = \frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$
- **Shoelace Formula (Area of Polygon):** $A = \frac{1}{2} |\sum_{i=1}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) + (x_n y_1 - x_1 y_n)|$
OBS: All vertices must be in order, either clockwise or counterclockwise

- **Pick's Theorem (Area of Lattice Polygons):** $A = I + \frac{B}{2} - 1$, where I is interior points and B is boundary points

3.2 Circles

- **Area:** πr^2
- **Circumference:** $2\pi r$
- **Equation:** $(x - h)^2 + (y - k)^2 = r^2$

3.3 Basic 2D Geometry

- **Distance Formula:** $\sqrt{\Delta x^2 + \Delta y^2}$
- **Vector Definition:** $\vec{v} = (x, y)$
- **Dot Product** ($\vec{a} \cdot \vec{b}$): $x_1x_2 + y_1y_2 = |\vec{a}||\vec{b}|\cos\theta$
Usage: Finding angles, checking perpendicularity ($\vec{a} \cdot \vec{b} = 0$)
- **Cross Product** ($\vec{a} \times \vec{b}$): $x_1y_2 - y_1x_2 = |\vec{a}||\vec{b}|\sin\theta$
Usage: 2D orientation, calculating signed area, collinearity ($\vec{a} \times \vec{b} = 0$)
- **Midpoint:** $M = (\frac{x_1+x_2}{2}, \frac{y_1+y_2}{2})$

3.4 Area of regular polygons

The area of a regular polygon can be calculated by constructing triangles from the center of the polygon to its vertices. The formula for the area of a regular polygon with n sides and a circumradius R is given by:

$$A = \frac{1}{2}nR^2 \sin\left(\frac{360^\circ}{n}\right)$$

Alternatively, if the apothem a (the distance from the center to the midpoint of a side) is known, the area can also be calculated using the formula:

$$A = \frac{1}{2}nsa$$

where s is the length of a side of the polygon.

3.5 List of areas for common regular polygons

Here are the area formulas for some common regular polygons:

- Equilateral triangle (3 sides): $A = \frac{\sqrt{3}}{4}s^2$
- Square (4 sides): $A = s^2$
- Regular pentagon (5 sides): $A = \frac{1}{4}\sqrt{5(5+2\sqrt{5})}s^2$
- Regular hexagon (6 sides): $A = \frac{3\sqrt{3}}{2}s^2$
- Regular octagon (8 sides): $A = 2(1 + \sqrt{2})s^2$

These formulas allow for the calculation of the area of regular polygons based on their side length or other relevant measurements.

3.6 Algorithms/Concepts

- **Convex Hull (Monotone Chain or Graham Scan):** Finds the smallest convex polygon containing all points. $O(n \log n)$. Monotone Chain is simpler to implement.
- **Line Segment Intersection:** Two segments AB and CD intersect iff A, B are on opposite sides of line CD AND C, D are on opposite sides of line AB . Check using cross products.
- **Point in Polygon:** Winding number or ray casting in $O(n)$. Count how many times a ray from the point crosses polygon edges; odd = inside.
- **Angle of vector:** Use $\text{atan2}(y, x)$ to find the angle in $(-\pi, \pi]$. Prefer cross/dot products over angles to avoid float precision issues.
- **Closest Pair of Points:** Divide and conquer in $O(n \log n)$. Split by median x , recurse, then check strip of width $2d$ around the dividing line (at most 8 candidate points per side).
- **Rotating Calipers:** Technique to solve problems on convex hulls (diameter, width, minimum bounding rectangle) in $O(n)$ after the hull is built. Maintain two antipodal points and rotate them around the hull.

4 Identities

5 Notes

6 Probability

6.1 Common Distributions

Distribution	PMF/PDF	E[X]	Var(X)	Use case
Bernoulli(p)	$P(1) = p, P(0) = 1 - p$	p	$p(1 - p)$	Single trial
Binomial(n, p)	$\binom{n}{k} p^k (1 - p)^{n-k}$	np	$np(1 - p)$	k successes in n trials
Geometric(p)	$(1 - p)^{k-1} p$	$\frac{1}{p}$	$\frac{1-p}{p^2}$	Trials until first success
Poisson(λ)	$\frac{\lambda^k e^{-\lambda}}{k!}$	λ	λ	Rare events
Uniform(a, b)	$\frac{1}{b-a}$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$	Continuous uniform

- **Geometric distribution:** The expected number of fair-coin flips until the first Head is $E[X] = 1/p = 2$. In competitive programming: “*expected number of times to roll a die until a 6*” = 6.
- **Coupon Collector Problem:** Expected number of draws (with replacement, uniform over n types) to collect all n types:

$$E[T] = n \sum_{k=1}^n \frac{1}{k} = n \cdot H_n \approx n \ln n$$

- **Negative Binomial:** Expected number of trials until r successes each with probability p :

$$E[T] = \frac{r}{p}$$

- **Geometric Probability (Continuous):** Probability of a point falling inside a region = $\frac{\text{area of region}}{\text{total area}}$. Common in problems involving random points in squares/circles.

6.2 Probability Fundamentals

- **Axioms:**

$$- 0 \leq P(A) \leq 1; P(\Omega) = 1; P(\emptyset) = 0$$

$$- \text{Union: } P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

$$- \text{Complement: } P(\bar{A}) = 1 - P(A)$$

- **Conditional Probability:** $P(A | B) = \frac{P(A \cap B)}{P(B)}$, provided $P(B) > 0$.
- **Independence:** A and B are independent iff $P(A \cap B) = P(A)P(B)$.
- **Law of Total Probability:** If $B_1, \dots, B_n$ partition Ω :

$$P(A) = \sum_{i=1}^n P(A | B_i) P(B_i)$$

- **Bayes' Theorem:**

$$P(B_i | A) = \frac{P(A | B_i) P(B_i)}{\sum_j P(A | B_j) P(B_j)}$$

- **Expected Value:** $E[X] = \sum_x x \cdot P(X = x)$ (discrete). Key property: **Linearity** — $E[X + Y] = E[X] + E[Y]$ holds *even when X and Y are dependent*.
- **Linearity of Expectation (ICPC trick):** To count the expected number of elements satisfying a property, sum the probability that each individual element satisfies it. Example: expected number of fixed points in a random permutation = $n \cdot \frac{1}{n} = 1$.
- **Variance:** $\text{Var}(X) = E[X^2] - E[X]^2 \geq 0$. For independent X, Y : $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$.
- **Markov Chains:** A system with states $s_1, \dots, s_k$ and transition probabilities $P_{ij} = P(\text{go to } s_j | \text{in } s_i)$. Transition matrix T satisfies T_{ij}^n = probability of reaching s_j from s_i in exactly n steps. Stationary distribution π : $\pi T = \pi$, $\sum \pi_i = 1$.

7 Math

7.1 System of Linear Equations

A system of linear equations $Ax = b$ (with A an $n \times n$ matrix) has:

- **Unique solution** if $\det(A) \neq 0$.

- **Infinitely many solutions** if $\det(A) = 0$ and the system is consistent.
- **No solution** if $\det(A) = 0$ and the system is inconsistent.

7.2 Linear System Solving

Gaussian Elimination with Partial Pivoting — $O(n^3)$. Augment to $[A \mid b]$ and apply row operations:

1. For column j : find row $i \geq j$ with the largest $|A_{ij}|$; swap rows i and j (partial pivoting for numerical stability).
2. Divide row j by the pivot A_{jj} .
3. Subtract appropriate multiples of row j from all other rows to zero out column j .

After elimination, x is read directly from the last column. If a pivot is zero (or near-zero), the matrix is singular and the system has no unique solution.

7.3 Polynomials

- **Definition:** A **polynomial** is an expression of the form

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0,$$

where $a_n, a_{n-1}, \dots, a_0$ are coefficients and n is a non-negative integer called the degree of the polynomial.

- **Root of a polynomial:** A **root** of a polynomial $P(x)$ is a value r such that $P(r) = 0$.
- **Fundamental Theorem of Algebra:** Every non-constant polynomial with complex coefficients has at least one complex root.
- **Factorization:** A polynomial of degree n can be factored into linear factors over the complex numbers, i.e., if $P(x)$ is a polynomial of degree n , then it can be expressed as

$$P(x) = a_n(x - r_1)(x - r_2) \cdots (x - r_n),$$

where $r_1, r_2, \dots, r_n$ are the roots of the polynomial (counting multiplicities).

7.4 Algorithms

- **Fast Fourier Transform (FFT):** An efficient algorithm to compute the discrete Fourier transform (DFT) and its inverse, which can be used for polynomial multiplication.
- **Lagrange Inversion Formula:** A formula that provides the coefficients of the inverse of a power series, which can be applied to polynomials.
- **Lagrange Interpolation:** A method for finding a polynomial that passes through a given set of points.
- **Newton's Method:** An iterative method for finding successively better approximations to the roots of a real-valued function, which can be applied to polynomials.

7.5 Graphs

- **Definition:** A graph $G = (V, E)$ has vertices V and edges E . **Directed** (digraph) edges have direction; **undirected** edges do not.
- **Handshaking Lemma:** $\sum_{v \in V} \deg(v) = 2|E|$. In a digraph, $\sum \deg^+(v) = \sum \deg^-(v) = |E|$.
- **Tree:** A connected acyclic undirected graph. n vertices $\Rightarrow$ exactly $n - 1$ edges. Any two vertices are connected by a unique path.
- **Bipartite Graph:** Vertex set splits into U and V with all edges between the two sides. A graph is bipartite iff it contains no odd cycle. Check with BFS/DFS 2-coloring.
- **Euler Path/Circuit:**
 - **Undirected:** Eulerian *circuit* exists iff graph is connected and all vertices have even degree. Eulerian *path* exists iff exactly 2 vertices have odd degree (start/end there).
 - **Directed:** Eulerian *circuit* exists iff graph is strongly connected and $\deg^+(v) = \deg^-(v)$ for all v . Eulerian *path* exists iff one vertex has $\deg^+ - \deg^- = 1$ (start) and one has $\deg^- - \deg^+ = 1$ (end), all others balanced.
- **Hamiltonian Path/Cycle:** Visits each vertex exactly once. NP-hard in general; solvable with bitmask DP in $O(2^N \cdot N^2)$ for small N .

- **Planar Graph:** Can be drawn without edge crossings. By Kuratowski's theorem, planar iff it contains no subdivision of K_5 or $K_{3,3}$. Euler's formula: $V - E + F = 2$ (connected planar graph).
- **Hall's Marriage Theorem:** A bipartite graph (U, V) has a perfect matching iff for every $S \subseteq U$, $|N(S)| \geq |S|$.
- **Mantel's Theorem:** Triangle-free graph on n vertices has at most $\lfloor n^2/4 \rfloor$ edges.
- **Four Color Theorem:** Any planar graph is 4-colorable. Bipartite graphs are 2-colorable. Cycle graphs: 2-colorable if even, 3-colorable if odd.

7.6 Numerical Algorithms

- **Newton's Method:** Finds roots of a differentiable function iteratively:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Converges quadratically near the root. Also used for computing integer square roots and modular inverses. May diverge if the initial guess is far from the root or $f'(x_n) \approx 0$.

- **Binary Search on Real Numbers (Ternary Search):** To find the minimum/maximum of a unimodal function on $[lo, hi]$, repeatedly evaluate at $m_1 = lo + (hi - lo)/3$ and $m_2 = hi - (hi - lo)/3$ and discard the worse third. Converges in $O(\log_{3/2} \frac{1}{\epsilon})$ iterations.
- **Simpson's Rule:** Approximates a definite integral using parabolic segments:

$$\int_a^b f(x) dx \approx \frac{b-a}{6} [f(a) + 4f(\frac{a+b}{2}) + f(b)]$$

For higher accuracy, apply adaptively: recurse when $|\text{simpson}(a, m) + \text{simpson}(m, b) - \text{simpson}(a, b)| > \epsilon$.

- **Trapezoidal Rule:** Simpler integration approximation using trapezoids:

$$\int_a^b f(x) dx \approx \frac{b-a}{2} [f(a) + f(b)]$$

Less accurate than Simpson's for the same step size; use Simpson's in competitive programming.

8 Bitwise

8.1 Techniques

- **Check Even/Odd:** $n \& 1$ can be used to check if a number is even (result is 0) or odd (result is 1).
- **Power of 2 check:** $(n \& \sim(n-1))$ returns true if n is a power of 2.
- **Count 1s:** `__builtin_popcount(n)` counts set bits in $O(1)$. Use `__builtin_popcountll(n)` for `long long`. Manually: loop $n \& (n-1)$ until $n = 0$, each iteration removes the rightmost set bit.
- **Isolate/Clear Rightmost Set Bit:** Isolate: $n \& (-n)$. Clear: $n \& (n-1)$.
- **GCC Built-ins ($O(1)$):** `__builtin_clz(n)` — count leading zeros; `__builtin_ctz(n)` — count trailing zeros; `__builtin_parity(n)` — parity of set bits (0 or 1); `__lg(n)` = $\lfloor \log_2 n \rfloor$. All undefined for $n = 0$.
- **Subset Generation:** Loop from 0 to $2^n - 1$; each integer's bits represent inclusion/exclusion. To iterate over all subsets of a bitmask m : `for(int s=m; s>0; s=(s-1)&m)`.
- **Lowest Set Bit index:** `__builtin_ctz(n)` gives the index of the lowest set bit directly.

8.2 Algebraic Properties

- **Addition Relationship:** $a + b = (a \oplus b) + 2 \cdot (a \wedge b)$
- **OR Relationship:** $a|b = (a \oplus b) + (a \wedge b)$
- **AND Relationship:** $a \wedge b = \frac{(a+b) - (a \oplus b)}{2}$
- **Range XOR:** The XOR of all numbers from 1 to n follows a pattern based on $n \pmod 4$:
 - If $n \pmod 4 = 0$, then XOR sum = n
 - If $n \pmod 4 = 1$, then XOR sum = 1
 - If $n \pmod 4 = 2$, then XOR sum = $n + 1$
 - If $n \pmod 4 = 3$, then XOR sum = 0

9 C++

9.1 Template

C++ template for competitive programming:

```
#include <bits/stdc++.h>

using namespace std;

using vi = vector<int>;
using ll = long long;
// #define int long long
using ii = pair<int,int>;
using graph = vector<vector<int>>>;

#define nl '\n'
#define pb push_back
#define all(x) (x).begin(), (x).end()
#define sz(x) (int)(x).size()
#define rep(i, a, b) for(int i = a ; i < (b) ; ++i)

const int INF = 1e9;
const ll LINF = 1e18;

void solve(){
    //Solution
}

signed main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    int t = 1;
    //cin >> t;
    while(t--) solve();

    return 0;
}
```

9.2 Debug template

To print debug information, you can use the following template:

```
// ===== DEBUG TEMPLATE =====
template<typename A, typename B>
ostream& operator<<(ostream &os, const pair<A, B> &p) {
    return os << '(' << p.first << ", " << p.second << ')';
}
```

```
}

template<typename T_container, typename T = typename enable_if<!is_same<
    T_container, string>::value, typename T_container::value_type>::type>
ostream& operator<<(ostream &os, const T_container &v) {
    os << '{'; string sep;
    for (const T &x : v) os << sep << x, sep = ", ";
    return os << '}';
}

void dbg_out() { cerr << endl; }
template<typename Head, typename... Tail>
void dbg_out(Head H, Tail... T) { cerr << ' ' << H; dbg_out(T...); }

#ifdef ONLINE_JUDGE
#define dbg(...) cerr << "[" << __VA_ARGS__ << "]:", dbg_out(__VA_ARGS__)
#else
#define dbg(...)
#endif
// =====
```

9.3 System optimization

For faster IO, we can use the following code:

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

The first line disables synchronization between C++ and C streams; the second unties `cin` from `cout` so flushing doesn't happen before each read. Together they can speed up IO by 10× or more. `cout.tie(nullptr)` is redundant here and can be omitted.

9.4 Pragmas GCC Optimization

Write these directives on the **first line** of your code (before any include) to force the compiler to perform more aggressive optimizations.:

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

9.5 Containers

- **std::vector:** Dynamic array. $O(1)$ random access; $O(1)$ amortized `push_back`; $O(N)$ insert/erase at arbitrary position.

- **std::deque:** Double-ended queue. $O(1)$ push/pop at both ends; $O(1)$ random access; $O(N)$ insert in middle.
- **std::stack / std::queue:** Adapters (LIFO/FIFO). $O(1)$ push, pop, top/front.
- **std::priority_queue:** Max-heap by default. $O(\log N)$ push/pop; $O(1)$ top. For min-heap: `priority_queue<int, vector<int>, greater<int>`.
- **std::set / std::map:** Red-black tree. $O(\log N)$ insert, erase, find. `set` keeps elements sorted; `map` keeps key-value pairs sorted by key. `lower_bound/upper_bound` work on them.
- **std::multiset / std::multimap:** Like `set/map` but allow duplicate keys. To erase a single copy: `s.erase(s.find(x))` (not `s.erase(x)`, which removes all copies).
- **std::unordered_set / std::unordered_map:** Hash table. $O(1)$ average insert/find/erase; $O(N)$ worst case. Vulnerable to hash collisions — use a custom hash or `std::map` if anti-hash tests are suspected.
- **Policy-based tree (pb_ds):** `__gnu_pbds::tree` supports $O(\log N)$ `order_of_key(x)` (rank of x) and `find_by_order(k)` (k -th element). Useful as an ordered set with rank queries.

```
// Policy-based ordered set
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef tree<int,null_type,less<int>,
            rb_tree_tag,
            tree_order_statistics_node_update> ordered_set;

ordered_set s;
s.insert(3); s.insert(1); s.insert(5);
s.order_of_key(3); // 1 (0-indexed rank)
*s.find_by_order(0); // 1 (smallest element)
```

9.6 Algorithms

- **std::sort:** Hybrid Introsort (Quicksort + Heapsort + Insertion sort). $O(N \log N)$ average and worst case. Use `stable_sort` to preserve relative order of equal elements.
- **std::lower_bound / upper_bound:** Binary search on a sorted range. $O(\log N)$. `lower_bound` returns iterator to first element $\geq x$; `upper_bound`

returns first element $> x$.

- **std::binary_search:** Returns `true` if x exists in a sorted range. $O(\log N)$.
- **std::next_permutation:** Generates next lexicographic permutation. Returns `false` when wrapping around. $O(N)$ per call. Sort the range first to iterate all permutations.
- **std::accumulate:** Computes the sum (or custom reduction) of a range. $O(N)$. Default start value is 0; pass 0LL if result may overflow `int`.
- **std::fill / iota:** `fill(a, a+n, val)` sets all elements to `val`. `iota(a, a+n, start)` fills with `start, start+1, ...`.
- **std::min_element / max_element:** Return iterator to min/max in a range. $O(N)$.
- **std::unique:** Removes consecutive duplicates. Use after `sort` to deduplicate: `v.erase(unique(all(v)), v.end())`.
- **std::reverse:** Reverses a range in-place. $O(N)$.
- **__gcd / __lcm:** `__gcd(a,b)` (C++14) or `gcd(a,b)` from `<numeric>` (C++17). $O(\log \min(a,b))$.

10 Number Theory

10.1 Modular Arithmetic & Number Theory

- **Properties of Modulo:**
 - Sum: $(a + b) \% m = (a \% m + b \% m) \% m$
 - Subtraction: $(a - b) \% m = (a \% m - b \% m + m) \% m$
 - Product: $(a \times b) \% m = (a \% m \times b \% m) \% m$
- **Modular Division:** $(a/b) \equiv (a \times b^{-1}) \pmod{m}$, where $b \times b^{-1} \equiv 1 \pmod{m}$. Valid if $\gcd(b, m) = 1$.
- **Euler's Totient Theorem:** If $\gcd(a, m) = 1$, then $a^{\phi(m)} \equiv 1 \pmod{m}$.
- **Fermat's Little Theorem:** If p is prime, $a^{p-1} \equiv 1 \pmod{p}$. Inverse: $a^{-1} \equiv a^{p-2} \pmod{p}$.
- **Power Reduction:** If p is prime: $a^b \equiv a^{b \bmod (p-1)} \pmod{p}$. General: $a^b \equiv a^{b \bmod \phi(m)} \pmod{m}$ (if $\gcd(a, m) = 1$).

- **Bézout's Identity:** For a, b , there exist x, y such that $ax + by = \gcd(a, b)$. If $\gcd(a, m) = 1$, use Extended Euclidean Algorithm to find $a^{-1} \pmod{m}$.
- **Chinese Remainder Theorem (CRT):** Given pairwise coprime moduli $m_1, \dots, m_k$ and remainders $r_1, \dots, r_k$, there exists a unique $x \pmod{M}$ (where $M = m_1 \cdots m_k$) such that $x \equiv r_i \pmod{m_i}$ for all i .

$$x = \sum_{i=1}^k r_i \cdot \frac{M}{m_i} \cdot \left(\frac{M}{m_i} \right)^{-1} \pmod{M}$$

For two moduli: if $x \equiv a \pmod{m}$ and $x \equiv b \pmod{n}$ with $\gcd(m, n) = 1$, then $x \equiv a + m \cdot ((b - a) \cdot m^{-1} \pmod{n}) \pmod{mn}$.

10.2 Divisors and Prime Factorization

For an integer $n > 1$, let its prime factorization be $n = p_1^{a_1} \cdot p_2^{a_2} \cdots p_k^{a_k}$.

- **Number of Divisors (τ):** The total count of divisors of n .

$$\tau(n) = \prod_{i=1}^k (a_i + 1)$$

- **Sum of Divisors (σ):** The sum of all positive divisors of n .

$$\sigma(n) = \prod_{i=1}^k \left(\frac{p_i^{a_i+1} - 1}{p_i - 1} \right)$$

- **Product of Divisors (P):** The product of all divisors of n .

$$P(n) = n^{\frac{\tau(n)}{2}}$$

Note: If n is a perfect square, $\tau(n)$ is odd, but the result remains an integer.

- **Euler's Totient Function (ϕ):** Number of integers $k \in [1, n]$ such that $\gcd(k, n) = 1$.

$$\phi(n) = n \prod_{i=1}^k \left(1 - \frac{1}{p_i} \right)$$

11 Counting Problems

11.1 Union-Exclusion Principle

The Union-Exclusion Principle is a fundamental counting technique used to find the number of elements in the union of multiple sets. It states that for any finite sets $A_1, A_2, \dots, A_n$, the number of elements in their union can be calculated as follows:

$$\begin{aligned} |A_1 \cup A_2 \cup \dots \cup A_n| &= \sum_{i=1}^n |A_i| \\ &\quad - \sum_{1 \leq i < j \leq n} |A_i \cap A_j| \\ &\quad + \sum_{1 \leq i < j < k \leq n} |A_i \cap A_j \cap A_k| \\ &\quad - \dots + (-1)^{n+1} |A_1 \cap A_2 \cap \dots \cap A_n| \end{aligned}$$

In simpler terms: add the sizes of intersections of an **odd** number of sets; subtract those of an **even** number of sets. Equivalently, iterate over all $2^n - 1$ non-empty subsets $S \subseteq \{A_1, \dots, A_n\}$ and add $(-1)^{|S|+1} |\bigcap_{i \in S} A_i|$.

Typical use: count elements satisfying at least one of n properties (complements: count those satisfying none). Common in problems involving divisibility, forbidden configurations, or surjective functions.

11.2 Permutation

- **Definition:** A permutation of a set is an arrangement of its members into a sequence or linear order. The number of permutations of n distinct objects is given by $n!$ (n factorial).
- **Permutations with Repetition:** If there are n objects where there are n_1 indistinguishable objects of one type, n_2 indistinguishable objects of another type, and so on, the number of distinct permutations is given by $\frac{n!}{n_1! \cdot n_2! \cdot \dots}$.
- **Circular Permutations:** The number of ways to arrange n distinct objects in a circle is given by $(n-1)!$, since one object can be fixed to break the circular symmetry.

11.3 Modular Inverse Factorial

1. Calculate $n! \pmod{p}$ for a given prime p
2. Compute the modular inverse of $n!$ using Fermat's Little Theorem, which states that if p is a prime and a is an integer not divisible by p , then $a^{p-1} \equiv 1 \pmod{p}$. Therefore, the modular inverse of $n!$ can be calculated as $(n!)^{p-2} \pmod{p}$.
3. Use the precomputed factorial values and their inverses to efficiently compute the modular inverse of $n!$ for multiple queries.

11.4 Combination

- **Definition:** Number of ways to choose r items from n (order doesn't matter):

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

- **Key identities:**

- Symmetry: $\binom{n}{r} = \binom{n}{n-r}$
- Pascal's recurrence: $\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r}$
- Vandermonde: $\binom{m+n}{r} = \sum_{k=0}^r \binom{m}{k} \binom{n}{r-k}$
- Hockey stick: $\sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}$

- **Combinations with Repetition:** Choose r from n with repetition allowed:

$$\binom{n+r-1}{r} = \frac{(n+r-1)!}{r!(n-1)!}$$

- **Catalan Numbers:** $C_n = \frac{1}{n+1} \binom{2n}{n}$. Count: valid bracket sequences of length $2n$, BSTs with n keys, triangulations of $(n+2)$ -gon, monotonic lattice paths. $C_0 = 1, C_1 = 1, C_2 = 2, C_3 = 5, C_4 = 14$.
- **Pascal's Triangle:** Entry (n, r) gives $\binom{n}{r}$. Build in $O(n^2)$ using the recurrence; precompute factorials + modular inverses for $O(1)$ per query.

12 Python

12.1 Strings

- **Slicing:** `s[start:end:step]` — extracts a substring. `s[::-1]` reverses the string. `s[::2]` takes every other character. Indices can be negative (`s[-1]` = last char).
- **split / join:** `s.split(sep)` splits into a list; `sep.join(lst)` joins a list back. `s.split()` (no arg) splits on any whitespace and removes empty tokens.
- **strip:** `s.strip()` removes leading/trailing whitespace. `s.lstrip(chars)` / `s.rstrip(chars)` remove specific characters from each side.
- **find / count:** `s.find(sub)` returns the first index or -1. `s.index(sub)` raises `ValueError` if not found. `s.count(sub)` counts non-overlapping occurrences.
- **startswith / endswith:** `s.startswith(prefix)` and `s.endswith(suffix)` accept a tuple of strings to check multiple options at once.
- **Case methods:** `s.upper()`, `s.lower()`, `s.swapcase()`, `s.title()`, `s.capitalize()`.
- **Predicates:** `s.isdigit()`, `s.isalpha()`, `s.isalnum()`, `s.isspace()`, `s.islower()`, `s.isupper()`.
- **replace:** `s.replace(old, new, count=-1)` replaces all occurrences by default; pass `count` to limit.
- **ord / chr:** `ord('a') = 97`, `chr(97) = 'a'`. Useful for converting characters to indices: `ord(c) - ord('a')`.
- **Repetition:** `'ab' * 3 = 'ababab'`. Useful for generating patterns or padding.
- **Membership:** `'sub' in s` checks substring in $O(n)$. `s.index(sub)` finds first occurrence.
- **Sorting characters:** `sorted(s)` returns a list of chars; `"".join(sorted(s))` gives the sorted string. Useful for anagram checks: `sorted(a) == sorted(b)`.

- **f-strings:** `f"{val:.2f}"` formats floats. `f"{n:0{w}b}"` formats as binary with zero-padding of width `w`.
- **zfill / center / ljust / rjust:** `'7'.zfill(3) = '007'`. `s.center(width, char)`, `s.ljust(width)`, `s.rjust(width)` for alignment.
- **translate / maketrans:** Efficient character-level substitution or removal.

```
s = "hello world"
```

```
# Slicing and reversing
print(s[6:])      # "world"
print(s[::-1])    # "dlrow olleh"

# Split and join
words = s.split()      # ["hello", "world"]
print("-".join(words)) # "hello-world"

# Character index
idx = ord('c') - ord('a') # 2

# Anagram check
print(sorted("listen") == sorted("silent")) # True

# Remove non-alpha characters
clean = ''.join(c for c in s if c.isalpha())

# Count occurrences (including overlapping)
def count_overlap(s, sub):
    return sum(s[i:i+len(sub)] == sub
               for i in range(len(s) - len(sub) + 1))

# translate: remove vowels
table = str.maketrans('', '', 'aeiou')
print(s.translate(table)) # "hll wrld"

# Format binary with width 8
print(f"{42:08b}") # "00101010"
```

12.2 Useful Modules

- **math.gcd / math.lcm:** `math.gcd(a, b)` and `math.lcm(a, b)` (Python 3.9+). `math.gcd` accepts multiple args in 3.9+: `math.gcd(*lst)`.
- **math.isqrt:** Integer square root in $O(1)$, no float precision issues. Safer than `int(math.sqrt(n))`.
- **math.comb / math.perm:** `math.comb(n, k) =  $\binom{n}{k}$` and `math.perm(n, k) =  $P(n, k)$` , exact integers, no precomputation needed.

- **collections.Counter:** Counts element frequencies. Supports arithmetic (+, -, &, |). `most_common(k)` returns top `k` elements.
- **collections.defaultdict:** Like a dict but returns a default value for missing keys. `defaultdict(list)`, `defaultdict(int)`, `defaultdict(set)`.
- **collections.deque:** Double-ended queue with $O(1)$ `appendleft` / `popleft`. Use instead of `list` for BFS queues. Supports `maxlen` for sliding windows.
- **heapq:** Min-heap by default. For max-heap, negate values: `push -x`, `pop` and negate result. `heapq.nlargest(k, lst)` and `nsmlallest(k, lst)` in $O(n \log k)$.
- **bisect:** `bisect_left(a, x)` / `bisect_right(a, x)` return insertion points in a sorted list in $O(\log n)$. `insort` inserts maintaining order.
- **itertools.permutations / combinations:** `permutations(lst, r)` and `combinations(lst, r)` generate lazily. `combinations_with_replacement` allows repeats.
- **itertools.product:** `product(A, B) =` Cartesian product. `product(A, repeat=k) =  $A^k$` . Replaces `k` nested loops.
- **itertools.accumulate:** `accumulate(lst)` builds prefix sums. Pass `func` for other operations (e.g., `max`, `operator.mul`).
- **fractions.Fraction:** Exact rational arithmetic. `Fraction(1, 3) + Fraction(1, 6) = Fraction(1, 2)`. Avoids floating point errors.
- **string module:** `string.ascii_lowercase`, `string.ascii_uppercase`, `string.digits` give ready-made character sets.

```
from collections import Counter, defaultdict, deque
from itertools import permutations, combinations, product, accumulate
import heapq, math, bisect
```

```
# --- Counter ---
freq = Counter("abracadabra")
print(freq.most_common(2)) # [('a', 5), ('b', 2)]
# Check anagram
Counter("listen") == Counter("silent") # True

# --- defaultdict ---
graph = defaultdict(list)
graph[0].append(1) # no KeyError

# --- deque (BFS) ---
```

```

q = deque([start])
while q:
    v = q.popleft()          # 0(1)

# --- heapq (max-heap trick) ---
heap = []
heapq.heappush(heap, -5)
top = -heapq.heappop(heap)   # 5

# --- bisect ---
a = [1, 3, 5, 7]
i = bisect.bisect_left(a, 4) # 2 (first pos >= 4)

# --- prefix sums ---
pre = list(accumulate([1,2,3,4])) # [1,3,6,10]

# --- itertools.product (k nested loops) ---
for x, y, z in product(range(3), repeat=3):
    pass # 27 combinations

# --- math ---
print(math.isqrt(10))        # 3 (safe integer sqrt)
print(math.comb(10, 3))      # 120

# --- Fraction ---
from fractions import Fraction
print(Fraction(1,3) + Fraction(1,6)) # 1/2

```

12.3 Built-ins & Tricks

- **Big integers:** Python integers have arbitrary precision — no overflow, no long long needed. `2**1000` works natively.
- **pow(base, exp, mod):** Built-in modular exponentiation in $O(\log \text{exp})$. Much faster than `(base**exp) % mod`.
- **divmod(a, b):** Returns `(a // b, a % b)` in one call.
- **int(s, base):** Converts string `s` in given base to int. `int('ff', 16) = 255`, `int('1010', 2) = 10`.
- **bin / oct / hex:** `bin(10) = '0b1010'`, `hex(255) = '0xff'`. Slice `[2:]` to remove prefix.
- **enumerate:** `enumerate(lst)` yields `(index, value)` pairs. Avoids `range(len(lst))`.

- **zip:** `zip(a, b)` pairs elements. `zip(*matrix)` transposes a matrix. `zip_longest` from `itertools` handles unequal lengths.
- **map / filter:** `list(map(int, input().split()))` reads a line of ints. `filter(func, lst)` filters in-place lazily.
- **any / all:** `any(cond(x) for x in lst)` short-circuits on first True. `all` short-circuits on first False.
- **sorted with key:** `sorted(lst, key=lambda x: -x)` sorts descending. `sorted(words, key=len)` sorts by length. Stable sort.
- **min / max with key:** `max(lst, key=lambda x: x[1])` finds element with max second component.
- **Multiple assignment:** `a, b = b, a` swaps without a temp variable. `a, *rest = lst` unpacks head and tail.
- **List comprehension:** `[f(x) for x in lst if cond(x)]` replaces most filter+map patterns. Faster than equivalent loops.
- **lru_cache:** `@functools.lru_cache(maxsize=None)` memoizes a function. Replaces manual dp dictionaries for recursive solutions.
- **Fast I/O:** Use `sys.stdin.read().split()` to read all input at once when input is large.
- **Global recursion limit:** Default is 1000. Use `sys.setrecursionlimit(300000)` for deep recursion.

```

import sys, functools
input = sys.stdin.readline # faster input

# Read line of ints
n, m = map(int, input().split())
arr = list(map(int, input().split()))

# Modular exponentiation
print(pow(2, 10**9, 10**9 + 7)) # 0(log exp)

# Transpose matrix
matrix = [[1,2,3],[4,5,6]]
T = list(map(list, zip(*matrix)))

# Memoized recursion
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0: return j
    if j == 0: return i

```

```

return min(dp(i-1, j) + 1,
           dp(i, j-1) + 1,
           dp(i-1, j-1) + (s[i-1] != t[j-1]))

# Unpack head/tail
first, *rest = arr

# Flat 2D list comprehension
flat = [x for row in matrix for x in row]

# Read all input at once
data = sys.stdin.read().split()
idx = 0
n = int(data[idx]); idx += 1

```

13 Constants

13.1 Time and memory limits

- **Time:** Roughly 10^8 – 10^9 simple operations per second in C++. Use the table below to estimate the maximum N for a given complexity within a 1–2 second limit.
- **Memory:** 256 MB $\approx 6.4 \times 10^7$ ints (4 B each) or 3.2×10^7 long longs (8 B each).

Complexity	Max N (1–2 s)
$O(N!)$	≤ 12
$O(2^N)$	≤ 25
$O(N^3)$	≤ 500
$O(N^2)$	$\leq 5\,000$
$O(N^2 \log N)$	$\leq 3\,000$
$O(N\sqrt{N})$	$\leq 10^6$
$O(N \log^2 N)$	$\leq 10^6$
$O(N \log N)$	$\leq 10^7$
$O(N)$	$\leq 10^8$
$O(\sqrt{N})$	$\leq 10^{14}$
$O(\log N)$	$\leq 10^{18}$

13.2 Mathematical

- **Pi (π):** Usually defined as $\text{acos}(-1.0)$, approximately equal to 3.14159. In C++ 20, it is available as `std::numbers::pi`.
- **Euler (e):** Usually defined as the base of the natural logarithm, approximately equal to 2.71828. In C++ 20, it is available as `std::numbers::e`.
- **Golden Ratio (ϕ):** Usually defined as $(1 + \sqrt{5}) / 2$, approximately equal to 1.61803. In C++ 20, it is available as `std::numbers::phi`.

13.3 Data type limits

- **Integer limits:** The typical 32 bit signed integer bounds are $(\approx \pm 2 \times 10^9)$ and is obtained by using the `INT_MAX` and `INT_MIN` constants, while the 64 bit signed integer bounds are $(\approx \pm 9 \times 10^{18})$ and is obtained by using the `LLONG_MAX` and `LLONG_MIN` constants.
- **Floating-point limits:** The typical 32 bit floating-point bounds are approximately $\pm 3.4 \times 10^{38}$ and is obtained by using the `FLT_MAX` and `FLT_MIN` constants, while the 64 bit floating-point bounds are approximately $\pm 1.7 \times 10^{308}$ and is obtained by using the `DBL_MAX` and `DBL_MIN` constants.